

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Редакционное расстояние
Вариант: 5а

Студентка гр. 4343

Попова Е.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание.

4.1.1 Редакционное расстояние (Вагнер-Фишер)

Над строкой ϵ (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции: $\text{replace}(\epsilon, a, b)$ – заменить символ a на символ b .

$\text{insert}(\epsilon, a)$ – вставить в строку символ a (на любую позицию).

$\text{delete}(\epsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки

A и B , а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B .

Входные данные: первая строка – три числа: цена операции replace , цена операции insert , цена операции delete ; вторая строка – A ; третья строка – B .

Выходные данные: одно число – минимальная стоимость операций.

4.1.2

Над строкой ϵ (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции: $\text{replace}(\epsilon, a, b)$ – заменить символ a на символ b .

$\text{insert}(\epsilon, a)$ – вставить в строку символ a (на любую позицию).

$\text{delete}(\epsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки A в строку B .

4.2 Редакционное расстояние (Левенштейн)

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: `pedestal` $\rightarrow$ `stal`.
- Затем необходимо заменить два последних символа: `stal` $\rightarrow$ `stie`.
- Потом нужно добавить символ в конец строки: `stie` $\rightarrow$ `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($S, 1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($T, 1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Выполнение работы.

Основные структуры данных

4.1:

- `dp_table` (`list[list[int]]`) – таблица динамического программирования размером $(n+1) \times (m+1)$, где `dp[i][j]` хранит минимальную стоимость преобразования первых i символов строки A в первые j символов строки B .
- `cost_replace` (`int`) – стоимость операции замены одного символа на другой.
- `cost_insert` (`int`) – стоимость операции вставки символа в строку.

- `cost_delete (int)` – стоимость операции удаления одного символа.
- `cost_delete_two (int)` – стоимость операции удаления двух последовательных разных символов (вариант 5а).
- `str_a (str)` – исходная строка A.
- `str_b (str)` – целевая строка B.

4.2:

- `prev_row (list[int])` – предыдущая строка таблицы DP при оптимизированной реализации. Используется для снижения потребления памяти до $O(\min(n,m))$.
- `curr_row (list[int])` – текущая строка таблицы DP при оптимизированной реализации.
- `str_s (str)` – исходная строка S.
- `str_t (str)` – целевая строка T.

Основные функции.

4.1.1

- `edit_distance_with_variant(costs, str_a, str_b)` – основная функция. Извлекает стоимости операций, при необходимости доопределяет `cost_delete_two` (если передано только три числа, то `cost_delete_two = cost_delete * 2`), создаёт таблицу DP и возвращает минимальную стоимость из `dp[n][m]`.
- `create_dp_table(n_val, m_val, str_a, str_b, cost_replace, cost_insert, cost_delete, cost_delete_two)` – создаёт таблицу динамического программирования размером $(n+1) \times (m+1)$, инициализирует первую строку и первый столбец, затем вызывает `fill_dp_table`.
- `init_first_row(dp_table, m_val, cost_insert)` – инициализация первой строки таблицы: преобразование пустой строки в префиксы строки B (стоимость j вставок).

- `init_first_col(dp_table, n_val, cost_delete)` – инициализация первого столбца: преобразование префиксов строки A в пустую строку (стоимость i удалений).
- `fill_dp_table(...)` – двойной цикл по всем ячейкам `dp[i][j]` (где i от 1 до n , j от 1 до m), для каждой ячейки вызывает `compute_cell`.
- `compute_cell(dp_table, i_idx, j_idx, str_a, str_b, cost_replace, cost_insert, cost_delete, cost_delete_two)` — вычисляет одну ячейку таблицы, рассматривая четыре возможных перехода:
 - замена или совпадение: $dp[i-1][j-1] + \text{cost_replace}$ (0 при совпадении);
 - вставка: $dp[i][j-1] + \text{cost_insert}$;
 - удаление одного символа: $dp[i-1][j] + \text{cost_delete}$;
 - удаление двух последовательных разных символов (вариант 5a): если $i \geq 2$ и $\text{str_a}[i-1] \neq \text{str_a}[i-2]$, то $dp[i-2][j] + \text{cost_delete_two}$.
 - Выбирается минимальное значение.
- `get_replace_cost(str_a, i_idx, str_b, j_idx, cost_replace)` – возвращает 0, если символы равны, иначе `cost_replace`.

4.1.2

- `edit_distance_with_operations(costs, str_a, str_b)` – заполняет таблицу DP (аналогично 4.1.1), затем вызывает `restore_operations` и возвращает список операций и минимальную стоимость.
- `restore_operations(dp_table, str_a, str_b, cost_replace, cost_insert, cost_delete, cost_delete_two)` — выполняет обратный проход от `dp[n][m]` к `dp[0][0]`. На каждом шаге определяет, какой переход дал оптимальное значение, и добавляет соответствующую операцию:
 - M – совпадение,
 - R – замена,
 - I – вставка,
 - D – удаление.

При использовании операции удаления двух символов добавляются два D.
В конце список операций разворачивается.

4.2

- `levenshtein_distance(s, t)` – основная функция вычисления расстояния Левенштейна с оптимизацией по памяти до $O(\min(n, m))$.

Принимает две строки `s` и `t`.

- `main()` – точка входа, считывающая все слова из стандартного потока ввода (`input().split()`). Если получено не менее двух слов, вызывает `levenshtein_distance(data[0], data[1])` и выводит результат.

Описание алгоритма

Работа программы начинается с инициализации: считываются стоимости операций (3 или 4 числа), исходная строка `A` и целевая строка `B`. Если передано 3 числа, стоимость удаления двух символов устанавливается как удвоенная стоимость удаления одного (`cost_delete_two = cost_delete * 2`), что обеспечивает совместимость с базовой задачей.

Фаза заполнения таблицы DP:

1. Инициализация базовых случаев:

- a. `dp[0][0] = 0` – преобразование пустой строки в пустую не требует операций.
- b. `dp[0][j] = dp[0][j-1] + cost_insert` – преобразование пустой строки в префикс `B` длины `j` требует `j` вставок.
- c. `dp[i][0] = dp[i-1][0] + cost_delete` – преобразование префикса `A` длины `i` в пустую строку требует `i` удалений.

2. Заполнение основной части таблицы:

Для каждой ячейки `dp[i][j]` (где `i` от 1 до `n`, `j` от 1 до `m`) рассматриваются четыре варианта перехода:

- a. Замена или совпадение: $dp[i-1][j-1] + \text{cost_replace}$ (, стоимость = 0, если символы равны)
 - b. Вставка: $dp[i][j-1] + \text{cost_insert}$ – вставка символа $B[j-1]$ в строку A.
 - c. Удаление одного символа: $dp[i-1][j] + \text{cost_delete}$ – удаление символа $A[i-1]$.
 - d. Удаление двух разных символов (вариант 5a): если $i \geq 2$ и $\text{str_a}[i-1] \neq \text{str_a}[i-2]$, то $dp[i-2][j] + \text{cost_delete_two}$
3. Выбор минимума: $dp[i][j] = \min(\text{все рассмотренные варианты})$
 4. Восстановление операций (4.1.2): После заполнения таблицы выполняется обратный проход от $dp[n][m]$ к $dp[0][0]$. На каждом шаге определяется, какой переход дал оптимальное значение, и добавляется соответствующая операция (M – совпадение, R – замена, I – вставка, D – удаление). При использовании операции удаления двух символов добавляются два 'D'.

Завершение: Ответ находится в ячейке $dp[n][m]$, где n и m – длины строк A и B соответственно. Для задачи 4.1.2 выводится последовательность операций, полученная обратным проходом.

4.1.2

После заполнения таблицы выполняется обратный проход от $dp[n][m]$ к $dp[0][0]$. На каждом шаге определяется, какой переход дал оптимальное значение, и добавляется соответствующая операция:

- M – совпадение,
- R – замена,
- I – вставка,
- D – удаление.

При использовании операции удаления двух символов добавляются два D, а указатели i и j уменьшаются на 2 и 0 соответственно. В конце список

операций разворачивается, так как обход шёл от конца строк к началу. Выводится строка операций.

Особенности задачи 4.2 (Левенштейн):

Для вычисления расстояния Левенштейна применяется оптимизация по памяти $O(\min(n,m))$. Вместо хранения полной таблицы $(n+1) \times (m+1)$ используются только две строки:

- `prev_row` – значения предыдущей строки таблицы DP
- `curr_row` – значения текущей строки

Алгоритм последовательно вычисляет каждую строку, переиспользуя память:

1. Если $n < m$, строки s и t меняются местами (это гарантирует, что массив имеет длину $\min(n, m) + 1$).
2. Инициализация: `prev_row = [0, 1, 2, ..., m]`
3. Для каждой строки i от 1 до n :
 - a. `curr_row[0] = i;`
 - b. для каждого j от 1 до m :
 - i. Если символы совпали: `curr_row[j] = prev_row[j-1]`
 - ii. Иначе: `curr_row[j] = 1 + min(prev_row[j], curr_row[j-1], prev_row[j-1])`
4. Переиспользование: `prev_row, curr_row = curr_row, prev_row` (меняются местами)
5. Ответ: `prev_row[m]` после обработки всех строк

Оптимизация алгоритма

Для 4.1

1. Разбиение на подфункции (только в 4.1.1): Каждая логическая часть алгоритма вынесена в отдельную функцию (инициализация

строк/столбцов, заполнений таблиц, вычисление ячейки, получение стоимости замены). Это снижает сложность и улучшает читаемость кода.

В 4.1.2 аналогичное разбиение не применялось – заполнение таблицы и восстановление операций сосредоточены в двух функциях.

2. Раннее вычисление стоимости замены (только в 4.1.1): Функция `get_replace_cost` инкапсулирует проверку равенства символов и делает код нагляднее.
3. Условное вычисление операции удаления двух символов: Проверка `i_idx >= 2 && str_a[i_idx-1] != str_a[i_idx-2]` выполняется только когда это необходимо, предотвращая лишние операции. Это исключает обращение к `dp[i-2]` при `i < 2` и предотвращает некорректное «удаление двух одинаковых» символов.
4. Необходимость полной таблицы (только для 4.1.2): В отличие от 4.1.1, где достаточно ответа `dp[n][m]`, для восстановления редакционного предписания нужна вся таблица $(n+1) \times (m+1)$: обратный проход идёт от `dp[n][m]` к `dp[0][0]` и на каждом шаге сравнивает текущую ячейку с её предшественниками. Поэтому оптимизация памяти до двух строк здесь неприменима.
5. Потенциальная оптимизация памяти (только для 4.1.1): Задача 4.1.1 выводит только число, поэтому теоретически можно отказаться от полной матрицы и хранить три строки (текущую, предыдущую и пред-предыдущую), что снижает память с $O(n \times m)$ до $O(\min(n, m))$. В данной реализации это не сделано – используется полная таблица.

Для 4.2

1. Предварительный обмен строк по длине. Если `len(s) < len(t)`, строки меняются местами, чтобы `t` была короче. Это гарантирует, что вспомогательные массивы имеют длину $\min(n, m) + 1$, что и даёт оценку памяти $O(\min(n, m))$

2. Оптимизация по памяти до $O(\min(n, m))$: Вместо полной таблицы $(n+1) \times (m+1)$ хранятся только две строки: `prev_row` и `curr_row`. После обработки очередной строки массивы меняются местами (`prev_row, curr_row = curr_row, prev_row`), а старая строка переиспользуется.
3. Единые стоимости операций: Все операции имеют стоимость 1, поэтому рекуррентное соотношение упрощается: при совпадении `curr_row[j] = prev_row[j-1]`, иначе `curr_row[j] = 1 + min(prev_row[j], curr_row[j-1], prev_row[j-1])`. Ветвлений по типам операций и их стоимостям нет – это ускоряет внутренний цикл.
4. Отсутствие вспомогательных функций: Логика инициализации и вычисления ячейки встроена непосредственно в `levenshtein_distance`. Это осознанный выбор в пользу компактности и производительности: в Python вызов функции на каждую ячейку заметно дороже, чем inline-вычисление.

Оценка сложности

Задачи 4.1.1, 4.1.2

Теоретическая сложность:

- Временная сложность: $O(n \times m)$, где n и m – длины строк A и B
 - Инициализация первой строки: $O(m)$
 - Инициализация первого столбца: $O(n)$
 - Заполнение таблицы: $O(n \times m)$, каждая ячейка вычисляется за $O(1)$
 - Восстановление операций (только 4.1.2): $O(n + m)$ – обратный проход по диагонали таблицы.
 - Итог: $O(n + m + n \times m) = O(n \times m)$.
- Пространственная сложность: $O(n \times m)$
 - Вспомогательные переменные: $O(1)$.
 - Для 4.1.1 теоретически возможно снижение до $O(\min(n, m))$ при хранении трёх строк (текущей, предыдущей и пред-предыдущей —

третья нужна из-за операции удаления двух символов). В данной реализации это не применено.

- Для 4.1.2 оптимизация памяти неприменима, так как обратный проход требует всей таблицы.

Практическая сложность:

Алгоритм выполняется за короткое время для строк длиной больше 2500 символов. Добавление четвёртой операции (удаление двух разных символов) не увеличивает асимптотическую сложность, так как проверка выполняется за $O(1)$ на ячейку.

Задача 4.2

Теоретическая сложность:

- Временная сложность: $O(n \times m)$, где n и m – длины строк A и B
 - Инициализация `prev_row`: $O(m)$.
 - Обработка каждой из n строк: $O(m)$ (внутренний цикл по j).
 - Итог: $O(n \times m)$.
- Пространственная сложность: $O(\min(n, m))$.
 - Хранятся два массива `prev_row` и `curr_row` длиной $\min(n, m) + 1$ каждый.
 - Предварительный обмен строк гарантирует, что длина массивов соответствует именно $\min(n, m)$, а не $\max(n, m)$.
 - Вспомогательные переменные: $O(1)$.
 - Экономия памяти по сравнению с полной матрицей – примерно в $\max(n, m)$ раз, что делает алгоритм практически применимым даже для строк максимальной длины.

Практическая сложность:

При длинах до 2500 символов время выполнения занимает несколько секунд

Оценка по памяти

Задачи 4.1.1 и 4.1.2

Потребление памяти определяется полной таблицей динамического программирования размером $(n+1) \times (m+1)$, что даёт асимптотическую оценку $O(n \times m)$.

При максимальных длинах строк $n = m = 2550$ таблица имеет примерно 6.25 миллионов ячеек. С учётом дополнительных расходов Python на хранение объектов и указателей в списках, общее потребление памяти составляет около 200-300 МБ.

Вспомогательные структуры:

- Строки `str_a`, `str_b`: $O(n + m)$ — порядка десятков килобайт.
- Локальные переменные стоимостей и индексы циклов: $O(1)$.

Задача 4.2:

За счёт хранения только двух строк (`prev_row`, `curr_row`) длиной $\min(n, m) + 1$ достигается оценка памяти $O(\min(n, m))$.

При $n = m = 2500$ длина каждого массива составляет 2501 элемент. Два массива занимают порядка нескольких сотен килобайт, что примерно в 1000 раз меньше, чем в задачах 4.1 с полной таблицей.

Вспомогательные структуры:

- `prev_row`, `curr_row`: $O(\min(n, m))$.
- Строки `s`, `t`: $O(n + m)$.
- Индексы циклов и временные переменные: $O(1)$.

Тестирование

Экспериментальные результаты подтверждают корректность работы алгоритма на различных тестовых случаях, включая:

- Строки разной длины
- Строки с одинаковыми символами
- Строки с полностью различными символами
- Пустые строки (граничный случай)
- Случай с использованием операции удаления двух разных символов

Таблица 1: Тестирование программы 4.1.1

Входные данные	Выходные данные
1 1 1 entrance reenterable	5
1 1 1 2 abc ac	1 (удаление 'b')
1 1 1 1 abcd ad	1 (удаление 'b' и 'c' одной операцией за 1)
5 3 3 av an	5 (замена 'v' на 'n')

Таблица 2: Тестирование программы 4.1.2

Входные данные	Выходные данные
1 1 1 abc ac	MDM abc ac (Совпадает a, Delete b, Match c)
1 1 1 1 abcd ad	MDDM abcd ad (Match a, Delete bc одной операцией, Match d)
1 1 1 cat cats	MMMI cat cats (Match c, a, t, Insert s)
1 1 1 abc abc	MMM abc abc (Всё совпадает)

Таблица 3: Тестирование программы 4.2

Входные данные	Выходные данные
pedestal stien	7
kitten sitting	3 (классический тест: 2 замены + 1 вставка)
abc abc	0 (строки совпадают)
abc abcd	1 (одна вставка)
flaw law	1 (одно удаление)

Вывод

В ходе выполнения лабораторной работы был успешно реализован алгоритм вычисления редакционного расстояния методом динамического программирования (алгоритм Вагнера-Фишера) включающий модификацию по варианту 5а: добавление четвёртой операции – удаления двух последовательных разных символов со своей стоимостью, а также восстановление последовательности операций для преобразования одной строки в другую (задача 4.1.2) и алгоритм вычисления расстояния Левенштейна с единичными стоимостями операций (задача 4.2), в котором применена оптимизация по памяти до $O(\min(n, m))$ за счёт хранения только двух строк таблицы DP (prev_row и curr_row) и обмена строк по длине в начале. В процессе разработки были использованы ключевые оптимизации: модульное разбиение алгоритма на подфункции с чёткой ответственностью (инициализация таблицы, вычисление ячеек, получение стоимости замены) в задаче 4.1.1, условная проверка операции удаления двух символов только при наличии достаточного количества символов и их различии, эффективное вычисление всех переходов за $O(1)$ на ячейку, а также оптимизация памяти в задаче 4.2, снижающая потребление с $O(n \times m)$ примерно до $O(\min(n, m))$. Отдельно отмечено, что в задаче 4.1.2 оптимизация памяти не может быть использована, поскольку обратный проход для

восстановления редакционного предписания требует всей таблицы $(n+1) \times (m+1)$, а для задачи 4.1.1 возможна потенциальная оптимизация до трёх строк (текущей, предыдущей и пред-предыдущей – третья необходима из-за операции удаления двух символов). Теоретическая сложность всех трёх задач составляет $O(n \times m)$ по времени, что подтверждает высокую практическую эффективность метода динамического программирования для решения задачи редакционного расстояния как с расширенным набором операций, так и с оптимизацией по памяти.